Code: Thực Hiện MC Dropout Bằng Keras

```
import numpy as np

# Bat tinh nang Training = True ben trong lop Dropout ke ca khi du doan
y_probas = np.stack([model(X_test_scaled, training=True)
                     for sample in range(100)])
y_proba = y_probas.mean(axis=0)
y_std = y_probas.std(axis=0) # Do bat dinh (Uncertainty)
```

Alpha Dropout

- Nếu bạn sử dụng hàm kích hoạt **SELU** để mạng tự chuẩn hóa (self-normalize).
- Thì sử dụng Dropout thông thường sẽ phá hủy sự tự chuẩn hóa đó!
- Hãy sử dụng `AlphaDropout` - một biến thể của Dropout bảo toàn được giá trị trung bình và phương sai.

```
model.add(keras.layers.AlphaDropout(rate=0.2))
```

Max-Norm Regularization

- Hạn chế (clamp) không cho chuẩn l2 (Norm 2) của vector trọng số của mỗi nơ-ron vượt quá một giá trị lớn nhất *r*.
- Cụ thể: $\|\mathbf{w}\|_2 \leq r$.
- Nếu vượt quá, trọng số sẽ được chia tỉ lệ: $\mathbf{w} \leftarrow \mathbf{w}\frac{r}{\|\mathbf{w}\|_2}$.
- Max-norm kết hợp rất tốt với Dropout để kìm hãm sự tăng trưởng vô hạn của các trọng số.

Code: Max-Norm Trong Keras

```
dense = keras.layers.Dense(100, activation="elu",
                           kernel_initializer="he_normal",
                           kernel_constraint=keras.constraints.max_norm(1.))
```

Tối ưu hóa RMSProp

- AdaGrad đôi khi giảm tốc độ học quá mức và dừng sớm trước khi đến mục tiêu cực trị toàn cục (global optimum).
- RMSProp là phiên bản nâng cấp của AdaGrad, giải quyết việc dừng sớm bằng cách chỉ tích lũy bình phương của các gradient **gần đây nhất** thay vì tất cả gradient từ đầu thuật toán.
- Thuật toán này sử dụng tốc độ phân rã lũy thừa β (exponential decay).

Code: Tối Ưu Hóa RMSProp Keras

```
# Tinh chinh toc do doc bang RMSprop
optimizer = keras.optimizers.RMSprop(learning_rate=0.001, rho=0.9)
```

- Tham số 'rho' tương ứng với β trong công thức lý thuyết (thường đặt là 0.9).
- Ngoại trừ những kiến trúc cực kỳ phức tạp, RMSProp gần như luôn hoạt động tốt hơn hẳn so với AdaGrad.

Adam, Nadam & AdamW

- **Adam** kết hợp Momentum (theo dõi động lượng) và RMSProp (theo dõi bình phương gradient).
- **Nadam** là Adam cộng với kỹ thuật Nesterov Accelerated Gradient (NAG), cho tốc độ hội tụ nhanh hơn một chút.
- **AdamW** là Adam kết hợp với kỹ thuật điều chuẩn phân rã trọng số (weight decay regularization) thực thụ, thường dùng cho mạng biến áp (Transformers).

Tóm Tắt Lựa Chọn Các Kỹ Thuật (Phần 1)

Thành phần	**Lựa chọn mặc định**
Cấu hình khởi tạo (Init)	Khởi tạo He (He initialization)
Hàm kích hoạt (Activation)	ELU hoặc GELU (hoặc Swish)
Chuẩn hóa (Normalization)	Mạng nông/sâu: Batch Normalization (BN)
Chống quá khớp (Regularize)	Early Stopping + Dropout
Trình tối ưu hóa (Optimizer)	Nadam hoặc AdamW (kèm Warmup)
Lịch trình tốc độ học	Performance Scheduling (hoặc One Cycle)

Tóm Tắt Lựa Chọn Nếu Tự Chuẩn Hóa (Phần 2)

Nếu bạn xây dựng Mạng dày đặc (Dense) và muốn nó tự chuẩn hóa (Self-Normalizing):

Thành phần	**Cấu hình Mạng Tự Chuẩn Hóa**
Cấu hình khởi tạo (Init)	Khởi tạo LeCun
Hàm kích hoạt (Activation)	SELU
Chuẩn hóa (Normalization)	Không cần (tự thân chuẩn hóa)
Chống quá khớp (Regularize)	Alpha Dropout

Các Vấn Đề Thực Tế Khác

- Nếu bạn làm việc trên các mô hình Thưa (Sparse models), hãy dùng L_1 Regularization.
- Nếu thiết bị của bạn cần tốc độ phản hồi cực nhanh (low latency) khi ứng dụng thực tế (Inference), hãy loại bỏ Batch Normalization bằng cách hợp nhất (fuse) các tham số vào lớp trước đó, và dùng Leaky ReLU.
- Luôn tận dụng tài nguyên có sẵn: Đừng quên Transfer Learning!

Tổng Kết Chương 11

- Chúng ta đã giải quyết rốt ráo việc **Huấn Luyện Các Mạng Khổng Lồ** mà không để rơi vào tình trạng Vanishing Gradient.
- **Khởi tạo He** kết hợp hàm **ELU / GELU** là lựa chọn tiêu chuẩn.
- Củng cố bằng **Batch Normalization**.
- Sử dụng **Adam Optimizer** với Lịch trình Học Tập (Learning Rate Schedule).
- Và đương nhiên, chống Overfitting hiệu quả bằng **Dropout**.